UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

Worksheet - 2

Aim: To implement Simple and Multiple Linear Regression using Python to predict continuous target variables based on different datasets from the UCI Machine Learning Repository.

Objective:

1. To download, load, and explore datasets using the pandas library.
2. To preprocess the data by splitting it into training and testing sets using scikit-learn.
3. To train a Linear Regression model and determine the mathematical equation of the best fit line (and hyperplane).
4. To evaluate the performance and accuracy of the trained model by calculating the Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).

Code:

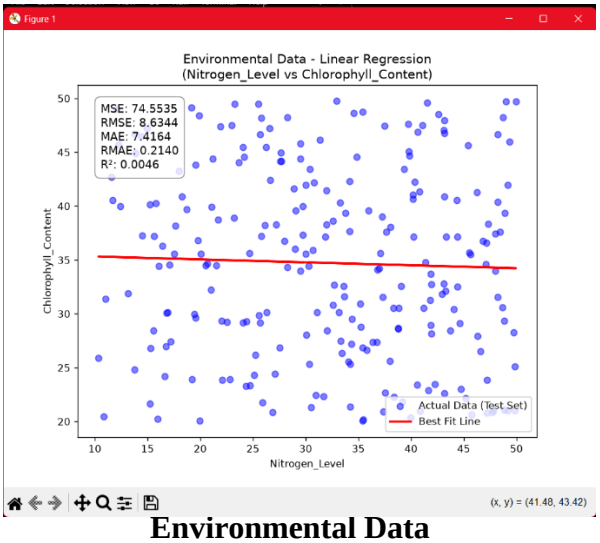
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split
import os

def run_linear_regression(csv_path, feature_col, target_col, output_plot_name, domain_name):
    print(f"--- Running {domain_name} Linear Regression ---")
    df = pd.read_csv(csv_path)
        if feature_col not in df.columns or target_col not in df.columns:
        print(f"Error: Columns {feature_col} or {target_col} not found in {csv_path}\n")
        return
        df = df.dropna(subset=[feature_col, target_col])
        X = df[[feature_col]]
    y = df[target_col]
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    model = LinearRegression()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
        mean_y = np.mean(y_test)
    rmae = mae / mean_y if mean_y != 0 else 0
```

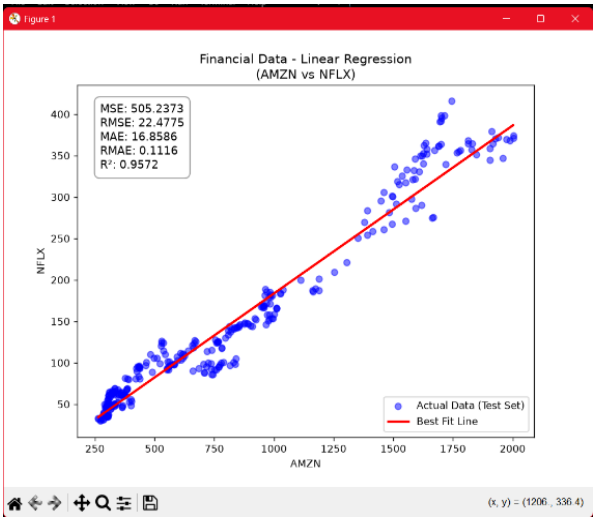
Saurav Bhuria (25MCA20133)
25MCA-6 (B)

```
        print(f"Domain: {domain_name}")
            print(f"Feature: {feature_col} -> Target: {target_col}")
            print(f"Best fit line equation: y = {model.coef_[0]:.4f} * x + {model.intercept_:.4f}")
            print(f"MSE: {mse:.4f} | RMSE: {rmse:.4f} | MAE: {mae:.4f} | RMAE: {rmae:.4f} | R2:
{r2:.4f}\n")
            plt.figure(figsize=(8, 6))
            plt.scatter(X_test, y_test, color='blue', alpha=0.5, label='Actual Data (Test Set)')
            plt.plot(X_test, y_pred, color='red', linewidth=2, label='Best Fit Line')
            plt.title(f'{domain_name} Data - Linear Regression\n({feature_col} vs {target_col})')
            plt.xlabel(feature_col)
            plt.ylabel(target_col)
            plt.legend(loc='lower right')
            textstr = '\n'.join((
                f'MSE: {mse:.4f}',
                f'RMSE: {rmse:.4f}',
                f'MAE: {mae:.4f}',
                f'RMAE: {rmae:.4f}',
                f'R²: {r2:.4f}'
            ))
            props = dict(boxstyle='round,pad=0.5', facecolor='white', alpha=0.8, edgecolor='gray')
            plt.gca().text(0.05, 0.95, textstr, transform=plt.gca().transAxes, fontsize=11,
                    verticalalignment='top', bbox=props)
            plt.savefig(output_plot_name)
            plt.show()
            plt.close()
        def main():
            base_dir = r"C:\Users\HP\.gemini\antigravity\brain\0ac317df-4884-4513-93ab-
cde763e5af5f\.user_uploaded"
            output_dir = r"C:\Users\HP\.gemini\antigravity\brain\0ac317df-4884-4513-93ab-cde763e5af5f"
                datasets = [
                (os.path.join(base_dir, "media__1785832776620.csv"), "Nitrogen_Level",
"Chlorophyll_Content", os.path.join(output_dir, "Environmental.png"), "Environmental"),
                (os.path.join(base_dir, "media__1785832784505.csv"), "AMZN", "NFLX",
os.path.join(output_dir, "Financial.png"), "Financial"),
                (os.path.join(base_dir, "media__1785832792155.csv"), "G1", "G3", os.path.join(output_dir,
"Educational.png"), "Educational"),
                (os.path.join(base_dir, "media__1785832798220.csv"), "Age", "Cholesterol_Level",
os.path.join(output_dir, "Medical.png"), "Medical")
            for csv_file, feat, target, plot_name, domain_name in datasets:
                run_linear_regression(csv_file, feat, target, plot_name, domain_name)
        if __name__ == "__main__":
            main()
```

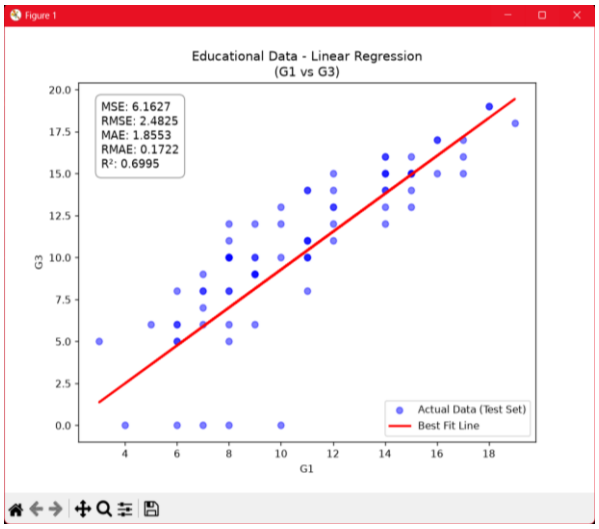
Output:



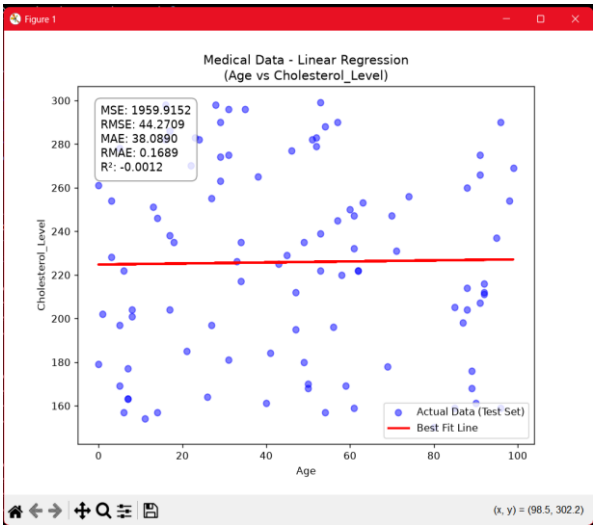
Environmental Data



Financial Data



Educational Data



Medical Data

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

[Done] exited with code=0 in 46.714 seconds

[Running] python -u "c:\Users\HP\OneDrive\Desktop\Linear_Regression.py"
--- Running Environmental Linear Regression ---
Domain: Environmental
Feature: Nitrogen_Level -> Target: Chlorophyll_Content
Best fit line equation: y = -0.0272 * x + 35.5780
MSE: 74.5535 | RMSE: 8.6344 | MAE: 7.4164 | RMAE: 0.2140 | R2: 0.0046

--- Running Financial Linear Regression ---
Domain: Financial
Feature: AMZN -> Target: NFLX
Best fit line equation: y = 0.2026 * x + -19.2080
MSE: 505.2373 | RMSE: 22.4775 | MAE: 16.8586 | RMAE: 0.1116 | R2: 0.9572

--- Running Educational Linear Regression ---
Domain: Educational
Feature: G1 -> Target: G3
Best fit line equation: y = 1.1295 * x + -2.0239
MSE: 6.1627 | RMSE: 2.4825 | MAE: 1.8553 | RMAE: 0.1722 | R2: 0.6995

--- Running Medical Linear Regression ---
Domain: Medical
Feature: Age -> Target: Cholesterol_Level
Best fit line equation: y = 0.0238 * x + 224.7008
MSE: 1959.9152 | RMSE: 44.2709 | MAE: 38.0890 | RMAE: 0.1689 | R2: -0.0012
```

Saurav Bhuria (25MCA20133)
25MCA-6 (B)

Linear Regression Performance Across Datasets

Model	MSE	RMSE	MAE	MAPE	sMAPE	R2
Environment	74.553	8.634	7.416	23.721	21.907	0.005
Financial	505.237	22.477	16.859	13.856	13.937	0.957
Educational	6.163	2.482	1.855	1.68e+17	29.255	0.699
Medical	1959.915	44.271	38.089	18.038	17.168	-0.001

Learning Outcomes:

1. Understand Theoretical Concepts: Grasp the underlying principles of Simple and Multiple Linear Regression and how they are used to model relationships between variables.
2. Implement Machine Learning Models: Gain hands-on experience using industry-standard Python libraries (like scikit-learn, pandas, and numpy) to build regression models.
3. Interpret Model Parameters: Understand how to extract and interpret the coefficients (slope) and intercept to form the best fit line equation.
4. Evaluate Model Performance: Effectively assess the predictive accuracy of a continuous model using standard evaluation metrics such as MSE and RMSE.

Saurav Bhuria (25MCA20133)
25MCA-6 (B)